

LC-CLOUD 要件定義書

2026/4/17

出版統制：POLARIS

目次

- 1. 背景と目的
- 2. スコープ
- 3. 全体設計原則
- 4. 機能要件（IAM／ポータル／コンピュート／ストレージ／ネットワーク／DevOps／運用・監視／周辺サービス）
- 5. 非機能要件（可用性／バックアップ／ログ／セキュリティ／コスト管理／運用）
- 6. 制約条件・前提
- 7. 技術基盤の評価軸
- 8. スケジュール・体制
- 9. 残課題（要確認事項）

改訂履歴

■ v0.1 / 2026-04-18 / 初版ドラフト（対話ベースの整理）

- ・ 利用者像、方向性、技術基盤の方針を確定
- ・ 機能要件A～H、非機能要件I～L、設計原則を整理

■ v1.0 / 2026-04-21 / 本書（PowerPoint版）

- ・ v0.1ドラフトを正式な要件定義書として体裁化
- ・ 残課題を第9章として明示

I. 背景と目的

■ 【背景】現状の社内クラウドの課題

- ・ ただのコンピュートリソース提供に留まり、利用者は環境を自力で組み上げる必要がある
- ・ 属人化が進み、運用ノウハウが特定メンバーに集中している
- ・ 非エンジニアにとっては利用ハードルが高く、シャドーITの温床となっている

■ 【目的】LC-Cloudで実現する姿

- ・ 承認済みサービスを統合的に提供する社内SaaSポータルへ進化させる
- ・ 非エンジニアを含む幅広い職種が、セルフサービスで必要なサービスを利用可能にする
- ・ API/IaCを基本としつつ、GUIで誰でも扱える三層UXを実現する

■ 【ビジョン】

- ・ 「ターミナルが開けない人でも、自分で必要なリソースを立ち上げられる」社内基盤

2. スコープ

■ 【対象範囲（In Scope）】

- ・ コンピュート（VM／GPU／コンテナ／Serverless）
- ・ ストレージ（S3互換／DBaaS／バックアップ）
- ・ ネットワーク（IP割当／DNS／LB等）
- ・ DevOps基盤（CI/CD／コンテナレジストリ）
- ・ アイデンティティ（SSO／IdP／MFA）
- ・ ポータル・コンソール（三層UX）
- ・ 監視・ログ収集
- ・ メールリレー、ドキュメント／ドライブ系（可否は別途決定）

■ 【対象外（Out of Scope）】

- ・ USBポートアクセス、2.4GHz AP等のオフィス物理ネットワーク領域
- ・ 個別の業務システム（人事・会計・SFA等）
- ・ エンドユーザーPC・VDI環境

3. 全体設計原則

■ API/IaC First

- ・ 全リソースはAPI経由で操作可能。GUIはその上の抽象化層に位置づける
- ・ Terraform Providerを第一級市民として提供する

■ 三層UX

- ・ 初心者／一般／管理者の利用者層ごとにコンソールの抽象度を変える
- ・ 権限と抽象度の両軸で設計する

■ SSOを全サービスの前提に

- ・ 個別認証・個別アカウントを絶対に作らない
- ・ 新規サービス追加時の審査基準にも含める

■ セルフサービス優先

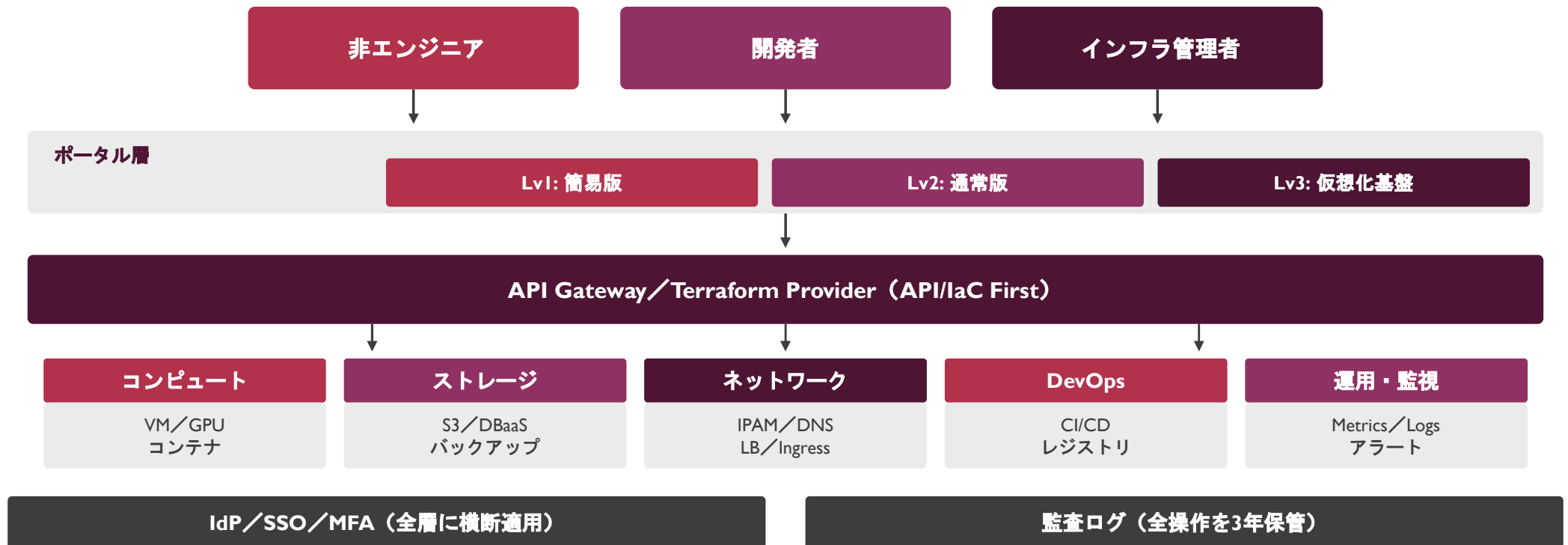
- ・ 申請→承認→払い出しの流れをすべて自動化（ワークフロー上で完結）

■ カタログ化による統制

- ・ 承認済みサービスのみをカタログに掲載し、シャドーITを抑止しつつ利用を促進する

4. 機能要件：システム全体構成

本章の各機能要件は以下の論理構成に位置づけられる。利用者は3層のUI経由で、共通APIを介して各サービスを利用する。



※ 詳細な機能要件は次スライド以降（4.1～4.8）を参照

4.1 機能要件：アイデンティティ・アクセス管理（IAM）

全サービスでSSO／MFAを必須とし、申請からアカウント発行までをワークフローで自動化する。

▼ アカウント発行フロー



▼ アカウントライフサイクル管理（要追加検討）

入社時	異動時	退職時	特殊
HRシステム連携での自動プロビジョニング	所属プロジェクト権限の自動更新	アカウント停止 所有リソースの引継ぎ	サービスアカウント ゲスト・APIトークン管理

4.2 機能要件：ポータル・コンソール（三層UX）

利用者の習熟度・権限に応じて3段階のコンソールを用意する。同一バックエンドAPIを共通化し、UIの抽象度を変える。

Level 1

簡易版コンソール

初心者向け
（非エンジニア）

- テンプレート選択 → 1クリックで払い出し
- 技術用語を隠蔽したGUI
- 申請フォーム形式の操作

Level 2

通常版コンソール

一般向け
（開発者）

- 標準的なリソース管理UI
- パラメータ指定可、ログ閲覧
- TerraformコードのImport対応

Level 3

仮想化基盤コンソール

管理者向け
（インフラ）

- Proxmox／oVirt／Horizon等の生UIへパススルー
- RBACで管理者ロールにのみ公開
- 全リソースの一括操作・トラブルシュート

※ いずれのLevelも同一のバックエンドAPI／Terraform Provider上で動作（API/laC First原則）

4.3 機能要件：コンピュータ

■ GPUインスタンス [Should]

- VRAM 24GB以上の選択肢（RTX 4090／A6000／H100クラスを想定）
- MIG（Multi-Instance GPU）対応も検討

■ GPUのスケジューリング [Should]

- キューイング＋タイムシェアリング機構（Slurm／Ray／Kueue等を検討）
- 予約制 or プリエンプティブ方式で需要超過に対応

■ Serverlessコンテナ実行基盤（オンプレCloud Run相当） [Could]

- Knative／KEDA／OpenFaaS等の活用。HTTPトリガで起動、アイドル時ゼロスケール

■ アプリ向けPaaS [Should]

- git push または Dockerfile 指定でデプロイできるシンプルなPaaS
- ハッカソン作品・個人プロダクトのデプロイ用途を想定（Dokku／Coolify等）

4.4 機能要件：ストレージ・データ

■ S3互換オブジェクトストレージ [Must]

- MinIO／Ceph RGW等を使用。バケット払い出し、アクセスキー発行

■ DBaaS [Must]

- PostgreSQL／MySQL／Redis等をマネージドで提供
- バックアップ・パッチ適用を自動化

■ VMバックアップ [Must]

- 全VMに対するスナップショットスケジュール機能
- 保持世代ポリシー、リストアUIを提供（詳細は5.2非機能要件参照）

■ ドライブサービス [Could]

- Nextcloud／Seafile等のホスティング、または SaaSカタログ経由で提供

4.5 機能要件：ネットワーク

■ AS活用とIP割当 [Should]

- ・ BGP接続済みの自社ASから、IPAM経由でグローバルIPv4／v6を申請ベースで割り当て
- ・ 逆引きDNS登録も含む

■ 内部ネットワーク設計 [要追加検討]

- ・ VPC／VLAN／サブネット分割方針
- ・ プライベート接続（社内他システム連携）

■ DNS管理 [Must]

- ・ ユーザーへFQDN払い出し機能

■ ロードバランサ／Ingress [Should]

- ・ ユーザーが利用可能な形でのLB／Ingressの提供

■ スコープ外

- ・ USBポートアクセス、2.4GHz AP等の物理層は本プロジェクトの対象外

4.6 機能要件：DevOps・開発者基盤

■ CI/CDパイプライン [Must]

- GitLab CI/Jenkins/Tekton/Argo Workflows 等の選定（要件確定後に決定）
- Runnerの払い出し、シークレット管理、アーティファクト保管

■ コンテナレジストリ [Must]

- Harbor/GitLab Container Registry 等を使用
- 脆弱性スキャン、イメージ署名、RBACによる権限制御

■ シークレット管理 [Must]

- HashiCorp Vault等の専用ストアに集約、平文格納禁止
- ローテーションポリシー、CI/CDからの参照方式を定義

4.7 機能要件：運用・監視

■ メトリクス収集 [Must]

- Prometheus／VictoriaMetrics等のメトリクスAPIを公開
- OpenTelemetry準拠。アラートはWebhook経由で外部連携可能

■ ログ収集 [Must]

- 監査ログ・アプリケーションログを一元的に収集（保持期間は5.3参照）

■ API／Terraform完全対応 [Must]

- 全操作をAPI／Terraformで実施可能とする（属人化解消の基本原則）
- 運用手順をコード化、ランブックもリポジトリ管理

■ サポート窓口・ステータスページ [要追加検討]

- チケット受付、対応SLA、対応時間帯の定義
- 計画停止・障害情報の告知用ステータスページ

4.8 機能要件：周辺サービス

■ メールサーバー（SMTPリレー） [Should]

- Postfix等の自前リレー、または外部（SendGrid／SES）への統一ゲートウェイ
- 送信権限はRBACで管理

■ ドキュメントツール [Could]

- esa代替候補：Outline／Wiki.js／HedgeDoc等の自前ホスティング
- またはSaaSカタログ経由で Notion／Confluence 等を提供（方針は要確認）

■ 通知サービス [要追加検討]

- Slack／Webhook／メール等への通知送信基盤

5.1 非機能要件：可用性

■ 基本方針：ベストエフォート提供

- ・ ユーザー向けSLA文書は作成しない
- ・ 停止時間や復旧時間に関する数値コミットはしない

■ コントロールプレーンの最低限の冗長化 [Must]

- ・ 対象：IdP、ポータル、API Gateway、課金システム、監査ログ収集基盤
- ・ 最低2ノード構成、DB冗長化（単一障害点を避ける）

■ ユーザリソースのHA化

- ・ ユーザー責任とする（必要なユーザーが自身で冗長構成を組む）

■ 災害復旧（DR）

- ・ RTO／RPO、拠点障害時の方針は別途定義（要追加検討）

5.2 非機能要件：バックアップ

対象種別ごとに保護方針を定める。ユーザーVMは選択式、PaaSは強制、コントロールプレーンは管理DBのみ。

対象	制御方式	バックアップ方式	世代/頻度	備考
ユーザーVM	オプトイン	スナップショット	3世代 (毎時/日次/週次選択)	払い出し時のチェックボックスで有効化
DBaaS	強制	DBダンプ/スナップショット	標準ポリシー	全PaaSで標準提供（要件J3）
S3互換ストレージ	強制	バケットバージョンング /レプリケーション	標準ポリシー	同上
コンテナレジストリ	強制	イメージレプリケーション	標準ポリシー	同上
ドライブ/ドキュメント	強制	ファイル履歴+ゴミ箱	標準ポリシー	全社利用サービスは確実に保護
コントロールプレーン	システム	管理DBのみ取得	システム規定	業務サービス用は対象外

※ 保存先は本番ストレージとは別領域。物理分離か論理分離かは要確認（残課題）

5.3 非機能要件：ログ保存

ログ種別ごとに保持期間と取り扱いを定める。監査ログは法令・調査要件から3年を確定。

ログ種別	対象内容	保持期間	備考・要件
監査ログ	認証イベント／API操作／管理操作	3年	改ざん耐性必須（WORM等）／優先度Must
アプリケーションログ	各サービスの動作ログ	TBD（例 90日）	要追加検討
システムメトリクス （高解像度）	Prometheus等の生データ	TBD（例 30日）	ストレージコストに影響
システムメトリクス （集約後）	ダウンサンプリング済みデータ	TBD（例 1年）	長期トレンド分析用
セキュリティイベント	侵入検知／異常検知ログ	TBD（要件確認）	監査ログとの関係を整理要

■ ログ収集基盤

Loki／Elasticsearch等を想定（製品選定は要件確定後に実施）

5.4 非機能要件：セキュリティ

■ 認証・認可 [Must]

- ・ SSO／MFAの全サービス適用（4.1参照）
- ・ RBACによる権限制御、最小権限の原則を徹底

■ 通信の暗号化 [Must]

- ・ 全面TLS化、内部通信もmTLS推奨

■ 保存データ暗号化 [Must]

- ・ ディスク暗号化、DB暗号化、オブジェクトストレージ暗号化
- ・ 鍵管理（KMS）の整備

■ シークレット管理 [Must]

- ・ Vault等の専用ストアに集約、平文格納禁止（4.6参照）

■ 脆弱性管理・PAM等 [要追加検討]

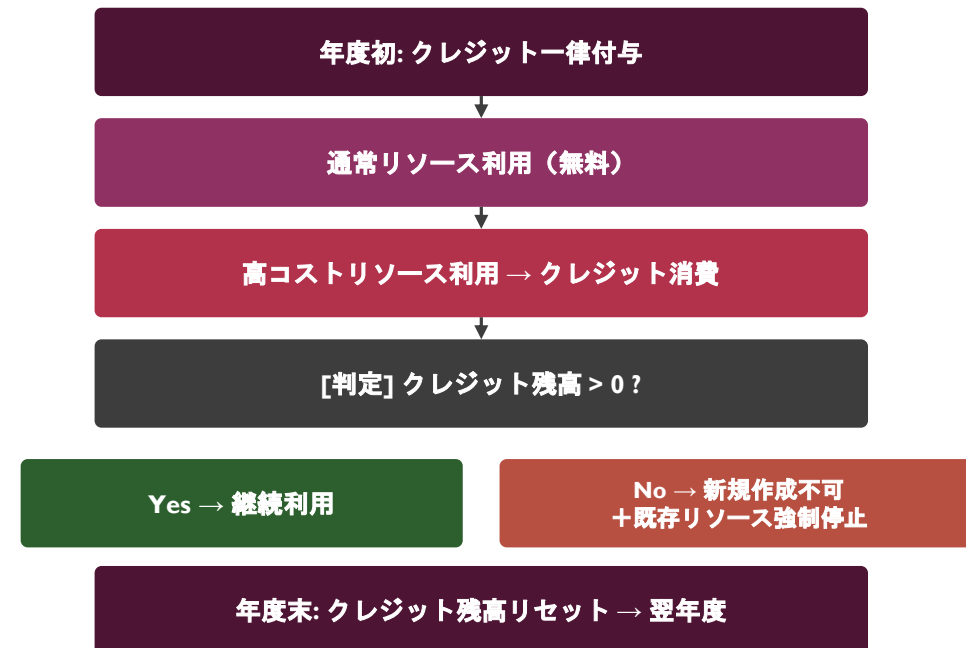
- ・ イメージスキャン頻度、CVE対応SLA、特権アクセス管理（PAM）

5.5 非機能要件：コスト管理（クレジット制）

原則無料＋クォータ＋クレジット制の3段階制御。GPU等の高コストリソースをクレジット消費とする。

項目	仕様
基本方針	大半のリソースは無料
クォータ	アカウントごとの種別別上限 （管理者により個別変更可）
消費対象	GPU等の高コストリソース （具体リストは要確認）
付与単位	個人単位
付与タイミング	年度初めに一律付与
繰り越し	なし（年度末リセット）
不足時挙動	新規作成不可 ＋既存は強制停止（削除はしない）

▼ クレジットライフサイクル



5.6 非機能要件：運用

■ コード化（Operations as Code） [Must]

- ・ 全操作はAPI／Terraformで実施可能
- ・ 運用手順のコード化を徹底（属人化解消の基本原則）

■ ランブック管理 [Must]

- ・ ランブックもリポジトリ管理し、変更履歴を追跡可能とする

■ メンテナンス・インシデント対応 [要追加検討]

- ・ メンテナンスウィンドウの定義、計画停止の告知方法
- ・ オンコール体制、夜間・休日の扱い
- ・ インシデント対応プロセス（検知→連絡→復旧→事後分析）

■ ドキュメント・トレーニング [要追加検討]

- ・ ユーザー向け／運用者向けドキュメントの整備
- ・ 特に非エンジニア向けのオンボーディング教材を用意

6. 制約条件・前提

■ 【制約条件】

- ・ 技術基盤（パブリック／プライベート／ハイブリッド）は本要件確定後に選定する
- ・ 既存社内クラウド（コンピュートリソース）からの移行を前提とする
- ・ 全社員が利用するため、UIは日本語必須（多言語対応の要否は別途検討）

■ 【前提条件】

- ・ 全社的にSSO基盤を導入する経営判断が前提（個別認証は禁止する方針）
- ・ 新規サービス追加には審査プロセスを経る運用体制が前提
- ・ クレジット制を支える課金エンジンは独立サブシステムとして開発する

7. 技術基盤の評価軸

■ 本要件の確定後に技術基盤の選定を実施する

■ 選定における比較軸

- ・ パブリッククラウドネイティブ（AWS／GCP／Azure） vs オンプレK8s／OpenStack vs ハイブリッド
- ・ 既存ツール活用（Backstage／AWS Service Catalog／Port／Terraform Cloud等） vs スクラッチ開発

■ 評価項目

- ・ 初期コスト・運用コスト
- ・ 拡張性・将来性
- ・ 社内エンジニアのスキルセットとの適合性
- ・ ベンダーロックインのリスク
- ・ セキュリティ要件・コンプライアンス要件への適合

8. スケジュール・体制

■ 【スケジュール】 ※TBD

- ・ 全体スケジュール、フェーズ分け（MVP→拡張→全社展開）は別途定義
- ・ リリース目標時期も合わせて確定する

■ 【体制】 ※TBD

- ・ プロジェクトオーナー、構築チーム、運用チームの構成と役割を別途定義
- ・ ステアリングコミッティ（意思決定の場）の設置検討
- ・ 変更管理プロセス（機能追加・変更時の承認ルート）の定義

■ 【既存システムからの移行】 ※TBD

- ・ データ・ユーザー・運用の引き継ぎ計画
- ・ 並行運用期間と既存システムの廃止時期

■ 【コミュニケーション】 ※TBD

- ・ 社内への周知・啓蒙計画、ユーザー向けトレーニング

9. 残課題（要確認事項）

■ 【スコープ関連】

- ・ ドキュメントツール（esa代替）の自前 vs SaaSカタログ方針
- ・ ドライブサービスの自前 vs SaaSカタログ方針

■ 【課金・クレジット関連】

- ・ クレジット消費対象リソースの具体リスト／単価／付与量
- ・ クレジット単位（円換算／独自ポイント／時間）

■ 【バックアップ・ログ関連】

- ・ バックアップ保存先の物理分離要否
- ・ 監査ログ以外の各種ログの保持期間／改ざん耐性の具体方式

■ 【機能仕様関連】

- ・ 三層コンソールのUI仕様詳細
- ・ 各ツール（IdP、ポータル、CI/CD、コンテナレジストリ等）の製品選定
- ・ GPUの需要超過時のキューイング方式

■ 【プロジェクト運営関連】

- ・ フェーズ分け／成功指標（KPI）／スケジュール／体制